AI-DLC Steering Doc Distribution

The Problem

Steering docs get copy-pasted into every repo. That means version drift, no single source of truth, and no clean way to update everyone at once. We want the canonical steering to live in one place, with each repo carrying a thin pointer instead of a copy.

The Constraint

The agents read local files. Claude Code reads `CLAUDE.md` and its `@import` chain from the local filesystem at session start. `@import` is `@path/to/file`, filesystem only. Cursor and Copilot work the same way against their own files. None of them natively fetch steering from a URL.

So the steering content has to be physically present locally at the moment the agent loads context. Nothing pulls from a remote source at read time. Whatever we build, the content lands locally first, then the agent reads it.

Why Not a Raw CDN

Serving steering off a CDN as "latest" is the `:latest` Docker tag problem applied to governance. A single edit would silently change agent behavior in every repo at once, including mid-stream on safety-critical work, with no review and no audit trail. For a regulated environment that is backwards.

We want the opposite: pinned, versioned, immutable, integrity-checked. A raw CDN makes you bolt all of that on by hand. A versioned artifact gives it to you for free.

The Model

Four parts.

A versioned artifact, not a CDN blob. The canonical layer (the AGENTS.md spine plus the domain chunks) is published to JFrog as a versioned package. An internal npm registry works the same way. Either gives versioning, immutability, checksums, and provenance out of the box.

A thin pinned manifest in the repo. The repo holds a small config that pins a version and lists which chunks it consumes. It does not hold the docs.

A tool-agnostic materialize step. A step that reads the manifest, pulls the pinned version, and writes it into a gitignored `steering/` dir. It runs outside any single agent, so one mechanism covers Claude Code, Cursor, and Copilot.

A CI guard. A check that the materialized copy still matches the pinned version, so nobody hand-edits the local copy and drifts.

The Components

JFrog package - the canonical steering, versioned. Spine plus domain chunks.

Repo manifest - pins the version, lists the chunks. Example:

```
{
    "steeringVersion": "2.4.1",
    "chunks": ["spine", "terraform", "dotnet", "pipeline"]
}
```

Materialize step - reads the manifest, pulls the pinned version into `steering/`, verifies the checksum. Runs as a devcontainer `postCreate`, a `make steering` target, or a `post-checkout` hook.

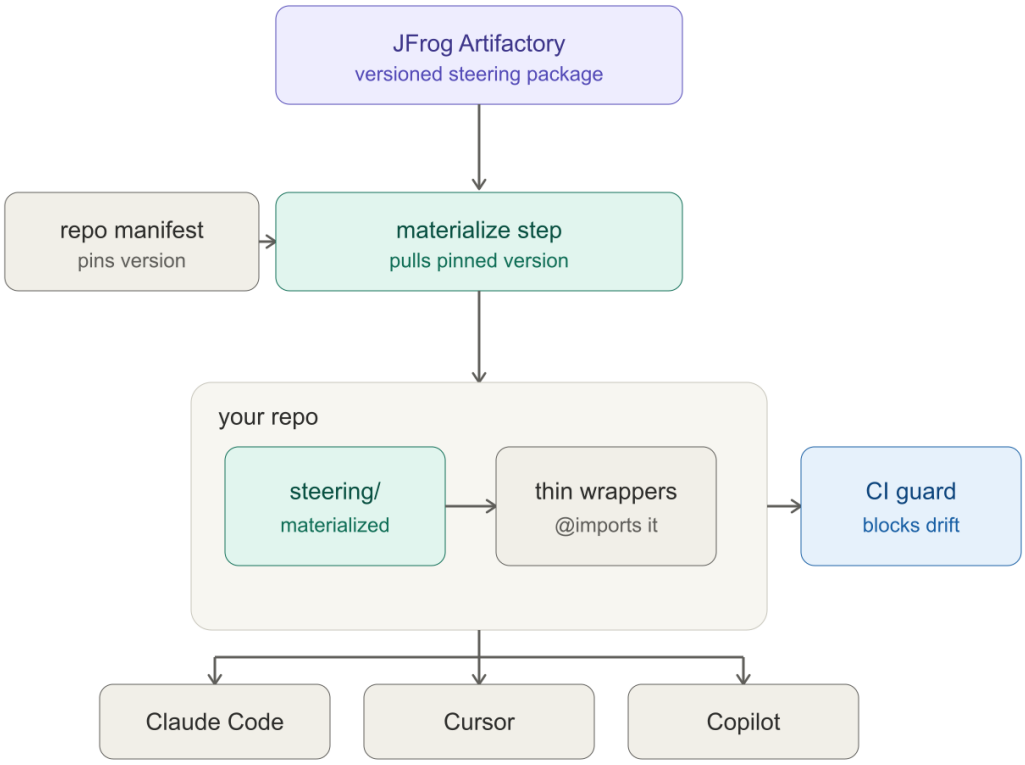
`steering/` dir - gitignored. The materialized files live here. Never hand-edited.

Thin wrappers - the per-platform files (`CLAUDE.md`, `.cursor/rules`, the Copilot instructions). They `@import` the materialized `steering/` files. They stay thin.

CI guard - validates that `steering/` matches the pinned version. Same deterministic gate pattern we trust everywhere else.

Agents - Claude Code, Cursor, Copilot. They read the wrappers, which pull in the shared steering. None of them fetch anything themselves.

The Flow



Steering distribution flow

The canonical steering lives once in JFrog as a versioned package. The repo carries only the thin manifest. The materialize step reads the pin, pulls the matching version into the gitignored `steering/` dir, and the wrappers `@import` it. All three agents read the wrappers, so they share one source. The CI guard validates that the local copy still matches the pin.

The Materialize Seam

This is the part that makes the whole thing tool-agnostic. The materialize step runs outside any agent - in the devcontainer build, a make target, or a git hook. Because it is not tied to Claude Code, Cursor, or Copilot, one step serves all three. That is the seam that lets us standardize once and have it apply everywhere.

For the Claude Code path specifically, a `SessionStart` hook can refresh the pinned version on session start as a convenience layer. It pulls a pinned version, falls back to the vendored copy when offline, and never becomes the source of truth. It sits on top of the materialize step, it does not replace it.

The Governance Loop

This is the payoff. Changing steering is a new version published to JFrog, then a manifest bump in each repo. That bump is a reviewable PR, scoped per repo. A safety-critical repo can sit on 2.4.1 while a sandbox repo runs 2.5.0-beta. With raw CDN latest you get none of that - one edit mutates everyone silently. Versioned artifact plus pinned manifest is the same conservative, auditable posture we took on MCP and direct integration.

TL;DR

The materialize step needs network access to JFrog at build or checkout time. Plan for offline and registry-down cases with a vendored fallback so an agent never silently loses its governance layer.

The wrappers stay thin on purpose. The moment real content creeps into a wrapper, it stops being a pointer and starts being a copy, and we are back to drift. The wrapper's only job is to `@import`.

The CI guard checks that the materialized copy matches the pin. It does not check that the pin is the right version - that is the manifest PR's job, and that is a human review.